Rust Verification Through Lean 4 Tooling Investigation report

Discovery report for Milestone 1 Ethereum

Delivered: February 02, 2026

runtime verification

Table of Contents

- Executive Summary
- Goals and Scope
  - Goals
- Tooling & Libraries
  - hax (Cryspen)
  - CompPoly
  - ArkLib
- PoC #1 — Merkle Root / Inclusion Verification (RISC Zero-inspired)
  - Goals this PoC adresses
  - Motivation and Source
  - Rust adaptation for extraction
  - Lean extraction and compilation strategy
  - Proof results (Merkle)
- PoC #2 — Horner Polynomial Evaluation (Plonky3-inspired)
  - Goals this PoC adresses
  - Motivation and Source
  - Rust preparation for extraction
  - Building hax + CompPoly together (Lean 4.26.0)
  - Deterministic patching for the Horner PoC
  - Proof results (Horner)
  - Spec-level correctness (CompPoly layer)
- Cross-project Bottleneck: Lean toolchain pinning mismatch
- Gaps and Required Upstream Work
  - hax gaps (for end-to-end correctness of extracted code)
  - CompPoly gaps (for fully mechanized Horner correctness without axioms)
- Issues and Pull Requests

    - hax
    - CompPoly
    - ArkLib
- Next Phase Plan
    - Conclusion

Executive Summary

This report describes the deliverables for Milestone/Phase 1.

Phase 1 – Exploration, Scoping, and Initial Extraction (Month 1)

During this phase, we successfully established reproducible development environments for the hax toolchain and conducted a systematic survey of zkEVM-related Rust repositories. We targeted several key components with significant overlap to ArkLib's and CompPoly's verification interests, including Horner evaluation implementation (inspired by Plonky3-style zkVMs) and Merkle tree structures (inspired by risk0). An initial round of extraction was performed using hax, which yielded concrete translation outputs and helped identify several feature gaps in the extraction pipeline. Additionally, we mapped relevant Rust APIs to corresponding CompPoly definitions, documenting alignment issues and potential opportunities for lemma reuse. This foundational work provided both a technical baseline and a clear roadmap for subsequent refinement and formalization.

This work investigates the feasibility of a practical Rust verification pipeline targeting zkVM / zkEVM components using Lean 4. We built and evaluated an end-to-end workflow:

Rust code → hax extraction → Lean 4 → specifications → proofs

We implemented two Proof-of-Concepts (PoCs):

1. Merkle-path root recomputation / verification (adapted from RISC Zero)
2. Horner polynomial evaluation (adapted from Plonky3-recursion)

Key outcomes:

- A working extraction pipeline was established using hax for both PoCs.
- For the Merkle PoC we proved basic properties about extracted verification logic (“compute-and-compare” shape + acceptance lemma).
- For the Horner PoC we achieved:
  - extracted code compilation under Lean 4.26.0 using deterministic patching steps (Core model patch + termination patch),

    - at least one proven “shape/behavior” lemma about extracted code,
    - a “spec-level” correctness proof connecting a CompPoly polynomial specification to a pure Horner evaluation function over $\mathbb{Z}$ (currently under explicitly stated assumptions / axioms due to missing automation lemmas).
- The work surfaced an ecosystem-level bottleneck: Lean toolchain pinning mismatch between hax and CompPoly. This led to cross-project coordination; maintainers converged on Lean 4.26.0 as a common target (hax update currently on a PR branch).
- Initial CI skeleton

Goals and Scope

Goals

- Identify practical constraints and gaps in verifying production-style Rust components extracted from zkVM-related codebases.
- Demonstrate minimal successful proofs for extracted code in Lean 4 (“documented basic proofs”).
- Identify where upstream issues/PRs are needed (hax / CompPoly / ArkLib).
- Environment setup scripts and initial CI skeleton.

Tooling & Libraries

hax (Cryspen)

- Used to extract Rust code to Lean 4.
- Lean backend is actively evolving; extraction frequently depends on the availability of core library models and termination/loop support.

CompPoly

- Lean library for computable multivariate polynomials.
- Intended as a specification backbone for polynomial-heavy components common in zk proof systems.

ArkLib

- Lean library of formalized cryptographic primitives/protocol abstractions.
- Initially planned as a spec target for crypto components; for this phase it mainly informed the modeling direction (oracle-style abstractions), while the Horner PoC focused on CompPoly specs.

PoC #1 — Merkle Root / Inclusion Verification (RISC Zero-inspired)

Goals this PoC adresses

- Target Identification: It successfully identifies and implements a concrete verification candidate: Merkle tree implementation (adapted from RISC Zero), selected for its relevance to zkVM stacks and compatibility with ArkLib's formalisms.
- Hax Exploration: It documents the iterative process of extraction.

Motivation and Source

We selected Merkle-path root recomputation / inclusion verification as a first candidate because:

- It is representative of zkVM stacks (commitments to memory/trace pages).
- It has a clear functional specification: fold over the authentication path.
- Correctness can be verified without cryptographic assumptions about the hash function.

Source reference (algorithmic inspiration):

- RISC Zero merkle.rs receipt logic (MerkleProof::root / verify).

Rust adaptation for extraction

We created a minimal Rust crate representing the core algorithm, adjusting the interface for extraction:

- removed trait objects (&dyn HashFn) in favor of a function parameter hash_pair,
- replaced error-handling style Result/ensure with a boolean or explicit comparison function,
- simplified data structures and dependencies to reduce extraction friction.

Lean extraction and compilation strategy

- Rust code extracted to Lean via cargo hax into lean.
- The extracted Lean code depended on core model namespaces that were incomplete; we introduced a local compatibility layer (“CoreCompat”) providing minimal stubs required for compilation.
- We also used a deterministic patch approach where necessary (e.g., noncomputable stubs in the early stages).

Proof results (Merkle)

We proved basic, machine-checked properties of the extracted verification logic (not crypto security):

- Theorem (shape): extracted verify is definitionally “compute root and compare with expected root”.
- Theorem (conditional acceptance): if the computed root evaluates to ok r, then verifying against r returns ok true.

These results demonstrate:

- extracted Rust code can be reasoned about in Lean 4,
- we can prove non-trivial properties even under incomplete core modeling by explicitly stating the modeling boundary.

PoC #2 — Horner Polynomial Evaluation (Plonky3-inspired)

Goals this PoC adresses

- Target Identification: It successfully identifies and implements a concrete verification candidate: polynomial evaluation (Horner's method), selected for its relevance to zkVM stacks and compatibility with CompPoly's formalisms.
- Hax Exploration: It documents the iterative process of extraction, from initial failures with loops to a working strategy using recursion, and records specific translation gaps (e.g., missing is_empty model, termination checker hurdles).
- Reproducibility: It establishes a reproducible environment by pinning specific toolchain versions (Lean 4.26.0, a Hax PR branch) and creating automated extraction scripts that include deterministic patches for the extracted code.
- Connection with Specification Libraries: It achieves an initial linkage by proving "shape" lemmas on the extracted Hax code and formulating a top-level correctness theorem that connects the extracted function's logic to a pure CompPoly specification (polyOfCoeffs and evalAt), while clearly mapping the remaining assumption boundary.

Motivation and Source

To align with CompPoly’s strengths, we selected polynomial evaluation:

- ubiquitous in zkVM/zk proof stacks (FRI, polynomial commitments, constraints),
- naturally specifiable with polynomial rings and eval,
- suitable for “Rust → CompPoly spec” linking.

Algorithmic reference:

- Plonky3-recursion evaluate_polynomial (Horner-style evaluation).

Rust preparation for extraction

We implemented an extraction-friendly version over i64.

Initial attempt:

- iterator-based reverse loop or while loop.
- hax Lean backend generates loop verification conditions requiring invariants; the extracted file did not typecheck due to missing invariant/VC scaffolding.

Mitigation:

- replaced the loop with a recursive helper (horner_rec). [Lean] extraction emits recursion over usize/machine ints without termination (brecOn missing):
- recursion over machine integers (usize/i64) fails Lean termination checking.

Final PoC strategy:

- keep recursion in Rust (to avoid iterator/VC loop obligations),
- patch extracted Lean to mark recursion as partial def (deterministic post-processing step).

Building hax + CompPoly together (Lean 4.26.0)

Status:

- CompPoly updated to Lean 4.26.0.
- hax Lean 4.26.0 support is available on a PR branch:
  - hax PR (not merged at time of writing)

- We built a dedicated PoC repository (horner-hax-comppoly-poc) using:
  - hax as a submodule pinned to the PR branch commit,
  - Lean toolchain pinned to leanprover/lean4:v4.26.0.

Deterministic patching for the Horner PoC

To make the extracted file compile, we implemented two deterministic patches:

- Core model patch: local definition of Core_models.Slice.Impl.is_empty (extracted code calls it; prelude had len but not is_empty).
- Termination patch: rewrite def horner_rec to partial def horner_rec in extracted Lean.

These patches are recorded in the extraction script (e.g., scripts/extract_horner.sh).

Proof results (Horner)

We achieved:

- extracted code compilation (under patched extraction),
- at least 2 proven “shape/behavior” lemmas for the extracted function:

Example (shape lemma):

```
    if Core_models.Slice.Impl.is_empty returns ok true, then horner_eval_i64 returns ok 0.
```

Spec-level correctness (CompPoly layer)

We formalized:

- polyOfCoeffs : List ℤ → CMvPolynomial 1 ℤ (Horner-form polynomial construction),
- evalAt : ℤ → CMvPolynomial 1 ℤ → ℤ via CMvPolynomial.eval.

We also defined a pure Horner evaluator over ℤ:

```
def hornerZ : List ℤ → ℤ → ℤ
| [], _ => 0
| c :: cs, x => c + x * hornerZ cs x
```

Goal theorem (spec-level correctness):

- evalAt x (polyOfCoeffs cs) = hornerZ cs x

Current status:

- this proof is implemented in SpecProof.lean under explicit assumptions (axioms) about eval_C and eval_X because upstream automation lemmas in CompPoly are not yet present/proved.

This provides a clear “assumption boundary”:

- once CompPoly supplies the missing simp/grind lemmas (or bridge lemmas), the proof becomes fully mechanized without axioms.

Cross-project Bottleneck: Lean toolchain pinning mismatch

A major finding: end-to-end “one project” integration was blocked when:

- CompPoly pinned Lean 4.22.0-rc2,
- hax pinned Lean 4.23, and did not support 4.22.

This was escalated through issues and maintainers discussion, and led to alignment:

- maintainers of hax, CompPoly, ArkLib agreed to converge on Lean 4.26.0.
- hax 4.26 support currently exists as PR #1880.

This coordination is a major success in enabling future end-to-end pipelines.

Gaps and Required Upstream Work

hax gaps (for end-to-end correctness of extracted code)

- Missing core model definitions referenced by extraction (example: Core_models.Slice.Impl.is_empty).
- Termination support for recursion over machine integers (usize, i64), or automatic emission of partial def / termination_by.
- Loop/invariant story: while extraction generates VCs; requires usable invariant annotations or improved backend defaults.
- Panic/overflow reasoning for +? / *? operations:
  - requires panic-freedom lemmas / bounded arithmetic assumptions.
- Semantic bridge i64 → ℤ under bounds.

CompPoly gaps (for fully mechanized Horner correctness without axioms)

These are primarily automation/normalization lemma gaps (simp/grind set):

1. @[simp]/@[grind] lemma(s) relating zero monomials to finsupp:
   - CMvMonomial.toFinsupp (0 : CMvMonomial n) = 0
   - (currently proofs get stuck on goals like Vector.get 0 i = 0).
2. simp-friendly lemmas about Unlawful / Std.ExtTreeMap lookup on empty and insert:
   - ∅[m]? = none
   - (insert k v)[k]? = some v
   - (insert k v)[m]? = none for m ≠ k
3. bridge lemmas between CompPoly and Mathlib representations:
   - fromCMvPolynomial (CMvPolynomial.C c) = MvPolynomial.C c
   - fromCMvPolynomial (CMvPolynomial.X i) = MvPolynomial.X i

4. final simp lemmas:
    - @[simp] eval_C and @[simp] eval_X

Issues and Pull Requests

hax

- Issue #1873 (toolchain mismatch / Lean version sensitivity):
  - Lean prelude doesn't build on Lean 4.22.0-rc2; case tags mismatch (inv1)
- PR #1880 (Lean 4.26 update; used for the Horner PoC; not merged at time of writing):
  - https://github.com/cryspen/hax/pull/1880
- Issue #1882 (termination / recursion / extracted recursion over usize;):
  - [Lean] extraction emits recursion over usize/machine ints without termination (brecOn missing)
- PR #1885 (Core_models.Slice.Impl.is_empty; merged):
  - https://github.com/cryspen/hax/pull/1885

CompPoly

- PR fixing README imports
  - https://github.com/Verified-zkEVM/CompPoly/pull/25
- PR adding eval_add / eval_mul simp/grind lemmas (merged)
  - https://github.com/Verified-zkEVM/CompPoly/pull/45
- Issue about C/X API mismatch (resolved/closed via PR)
  - https://github.com/Verified-zkEVM/CompPoly/pull/44
- Issue about toolchain bump / alignment to support hax integration
  - https://github.com/Verified-zkEVM/CompPoly/pull/46
    Current (in-progress) PR attempt: eval_C / eval_X automation

ArkLib

- Toolchain alignment discussion/coordination: in-progress

Next Phase Plan

1. Remove PoC patching where possible:
    - upstream the missing core-model definitions in hax prelude,
    - improve termination handling (or standardize partial def emission / termination measures).

2. Complete CompPoly automation layer:
    - land eval_C, eval_X and required bridge lemmas,
    - add additional simp/grind lemma packs for polynomial reasoning.

3. Finish end-to-end Horner correctness:
    - prove panic-freedom / overflow bounds for extracted i64 Horner,
    - prove integer-bridge lemma i64 → $\mathbb{Z}$ under bounds,
    - connect extracted result to hornerZ, then to CompPoly.evalAt.

4. Scale the target:
    - additional Plonky3 polynomial components: interpolation, multilinear extensions, quotient polynomials, etc.
    - prioritize targets where CompPoly already has a strong spec footprint.

Conclusion

In conclusion, the exploration and scoping phase successfully laid the necessary groundwork by establishing a reproducible environment for hax and systematically reviewing relevant Rust-based zkEVM components. Key targets included Horner evaluation routines—drawn from Plonky3-style zkVMs—and Merkle tree structures, aligning with CompPoly's formal verification priorities. Preliminary extractions via hax produced actionable translation outputs and surfaced several gaps in the extraction workflow. Furthermore, by mapping core Rust APIs to their CompPoly counterparts, we documented alignment challenges and identified opportunities for lemma reuse. This

initial stage not only provided a solid technical baseline but also established a clear, actionable direction for the refinement and formalization work to follow.